SCI Peer-Review Defense & Rebuttal Playbook (15 Questions — 4 Nov Revision)

Manuscript: Governing Human–AI Collaborative Project Workflows in Multi-Platform Digital Delivery: A Design Science Study of Auditable Handoffs and Idempotent Release Operations
Author: HuaiCheng Sheng (Agentic Work Lab, Hangzhou Chengxin Holding Group Co., Ltd. | ORCID: `0009-0003-2900-6377`)

This playbook is 100% synchronized with the revised 24-page manuscript (50 references, 11 tables, 5 figures) and `paper/reproduce_paper_tables.mjs`, providing ready-to-copy English responses and Chinese strategic summaries.

Q1 [Reviewer #1 (Project Governance & Stage-Gate Theory)]
EN: How does your four-layer architecture advance Stage-Gate project governance theory rather than merely packaging standard DevOps CI/CD scripts?
CN: 你们的四层架构是如何在理论上推进项目管理“阶段门控（Stage-Gate）”理论的，而不是仅仅封装了几个常规的 DevOps 脚本?

Standard SCI Rebuttal (English): As formalized in Section 2.4 ('Reading Stage-Gate Control as a Commit Protocol') and Section 7.1, classical Stage-Gate models (Cooper, 1990; Too & Weaver, 2014; Conforto & Amaral, 2016; Bianchi et al., 2022) assume human committees with shared organizational memory, while standard CI/CD pipelines (Humble & Farley, 2010) assume idempotent internal deployment targets. Multi-agent digital delivery violates both assumptions: autonomous LLM workflows lack durable cross-session state, and external publishing platforms expose non-idempotent endpoints without client-supplied idempotency keys (Helland, 2012). We bridge project governance and distributed commit protocols (Gray & Reuter, 1992; Saltzer et al., 1984; Garcia-Molina & Salem, 1987; Bainbridge, 1983) through five correspondences: (1) gatekeeping as deterministic admission control (planRun, Algorithm 1), (2) configuration baselines (Whyte et al., 2016) as cryptographic content addresses (--confirm CURRENT_PLAN_HASH), (3) the go decision as a write-ahead at-most-once commit (persisting UNKNOWN and final_submit_attempted = true before pressing Submit, Proposition 1), (4) post-gate review as an end-to-end read-back check, and (5) human escalation (WAITING_FOR_USER) as resolution of in-doubt transactions.

核心防御逻辑：第 2.4 节与第 7.1 节提出了“将阶段门控重构为分布式提交协议（Reading Stage-Gate Control as a Commit Protocol）”的理论综合。外部社交平台缺乏幂等键（Helland, 2012），一旦超时重发就会造成不可逆的重复发帖。我们通过 5 组理论映射将 Cooper (1990) 与 Too & Weaver (2014) 的项目治理理论同 Gray & Reuter (1992) 的预写日志（Write-Ahead）提交协议、Saltzer et al. (1984) 端到端原则及 Garcia-Molina & Salem (1987) Saga 补偿理论融为一体。

Q2 [Reviewer #1 (Human-in-the-Loop Governance & Ironies of Automation)]
EN: Why is a human-gated topology necessary when frameworks like AutoGen, MetaGPT, and ChatDev advocate fully autonomous multi-agent execution?
CN: 既然 AutoGen、MetaGPT 和 ChatDev 等框架都在主打全自动多智能体协作，为什么你们还要保留人类门控与接管?

Standard SCI Rebuttal (English): Conversational frameworks (AutoGen, MetaGPT, ChatDev; Table 1) operate inside local code sandboxes where errors can be rolled back via git revert. Live multi-platform publishing mutates public external state that cannot be rolled back and where delete-and-repost destroys work identity and URLs (Garcia-Molina & Salem, 1987). Drawing on Bainbridge's (1983) 'Ironies of Automation', Parasuraman et al. (2000), Shneiderman (2020), and Raisch & Krakowski (2021), automating routine preparation concentrates the remaining human role on

precisely the in-doubt or interactive boundaries where automation cannot safely guess (expired platform sessions, CAPTCHAs, or post-submit read-back timeouts). Designing explicit WAITING_FOR_USER and UNKNOWN states turns human intervention into a bounded, auditable transaction resolution step.

核心防御逻辑：AutoGen/ChatDev 是在本地沙盒写代码，错了能 git revert；公网发视频一旦连发或删错就是不可逆的公开事故。引用自动化经典文献 Bainbridge (1983)《自动化的讽刺》与 Shneiderman (2020)，常规步骤越自动化，留给去处理的边缘异常（扫码登录、验证码、提交后超时）就越关键。显式设计 WAITING_FOR_USER 与 UNKNOWN 状态，把人工介入变成了受界、可审计的"存疑事务裁决"。

Q3 [Reviewer #1 (DSRM, Action Design Research & Researcher Role)]
EN: Since the author served as both system designer and sole operator during the nine-day field deployment, how do you address researcher-role bias and methodological rigor?
CN: 既然作者在 9 天现场部署中同时担任系统设计者与唯一操作员，如何应对研究者角色偏差并保证设计科学的方法学严谨性?

Standard SCI Rebuttal (English): Sections 3.1, 3.3, and 7.4 explicitly situate this study within Design Science Research (Hevner et al., 2004; Peffers et al., 2007; Gregor & Hevner, 2013), Action Design Research (Sein et al., 2011), and Technical Action Research (Wieringa, 2014; Venable et al., 2016). To mitigate single-researcher bias, every empirical claim is anchored in contemporaneous machine-generated receipts committed to two public repositories (agentic-work-lab at data commit e3205d6 and publishops-crosscast at code commit 75b7abc) and recomputed deterministically by paper/reproduce_paper_tables.mjs. Furthermore, Section 7.4 explicitly discloses the dual designer-operator role and separates field observations, automated test results, and analytical baseline comparisons (Table 11).

核心防御逻辑：第 3.1、3.3 与 7.4 节依据 Sein et al. (2011) 的行动设计研究（ADR）与 Wieringa (2014) 的技术行动研究规范，坦诚披露了"设计者兼操作员"的双重角色，并通过完全公开的双仓库原始机器回执（数据快照 e3205d6、代码快照 75b7abc）与零依赖复算脚本 paper/reproduce_paper_tables.mjs 消除主观叙述偏差。

Q4 [Reviewer #2 (Distributed Systems & Proposition 1 Safety Guarantee)]
EN: In Proposition 1 and Algorithm 2, why is the receipt saved with state UNKNOWN and final_submit_attempted = true BEFORE pressing the final submit button, and what is the exact exception (A4)?
CN: 在命题 1（Proposition 1）与算法 2 中，为什么要在按下最终提交按钮之前先把状态 UNKNOWN 和 final_submit_attempted = true 落盘？例外条款 (A4) 又是什么?

Standard SCI Rebuttal (English): Without platform-side idempotency keys, a client cannot achieve exactly-once delivery on its own; it must choose between at-least-once and at-most-once (Helland, 2012; Kleppmann, 2017). In publishops-crosscast (browser_backends.py at commit 75b7abc), the adapter writes the receipt with state UNKNOWN and final_submit_attempted = true immediately BEFORE invoking the single final submit press ('Persist BEFORE clicking. A timeout must never cause another click'). Under Proposition 1 (Assumptions A1–A4), if a crash or timeout occurs during or after the press, any subsequent run sees final_submit_attempted = true and refuses to press again, with one explicit exception implemented in code (A4): a resumed run may press again only when the platform itself shows that the earlier work is still an unsubmitted draft. Thus Proposition 1 provides a formal safety guarantee (at-most-once automated submission) rather than a liveness guarantee.

核心防御逻辑：对应 browser_backends.py 的真实注释与代码（"Persist BEFORE clicking"）：在点击提交按钮前先行把 UNKNOWN 和 final_submit_attempted = true 原子落盘。命题 1 严谨列出了假设 A1–A4，并如实说明了代码中的唯一例外 (A4)：仅当远端平台明确显示此前作品仍处于"未提交草稿（unsubmitted draft）"时才允许恢复点击；同时明确指出这是安全性保证（Safety，最多提交一次）而非活性保证（Liveness）。

Q5 [Reviewer #2 (Field Evidence of Crash & Timeout Recovery)]
EN: What concrete field evidence supports your release state machine beyond unit tests?
CN: 除了单元测试之外，有哪些具体的现场实测证据支撑你们的发布回执状态机?

Standard SCI Rebuttal (English): Section 6.3 and Tables 5–6 document three concrete operational traces from the repositories: (1) during the five-platform release of the flagship film, the Douyin adapter completed upload and submission but timed out during post-submit verification, latching the receipt into UNKNOWN (with states PREPARING -> UNKNOWN -> SUBMISSION_VERIFIED / SUBMITTED_UNVERIFIED / WAITING_FOR_USER in Figure 2) without re-uploading; (2) a separate Douyin scheduled-card test on 1 November exhibited the same read-back timeout and likewise held without duplicate submission; and (3) during a private YouTube upload check on 30 October on the v0.15.1 API upload route (outside the evaluated 75b7abc browser snapshot), an upload forced to exit twice—first after 8,388,608 acknowledged bytes and then in a second process after saving the video ID—completed its thumbnail in a third process and kept the same video ID rather than creating a duplicate upload.

核心防御逻辑：第 6.3 节与表 5–6 给出了 3 个硬核实测证据：① 五平台发布中抖音出现回读超时，状态机安全锁入 UNKNOWN 未重传；② 11 月 1 日抖音定时卡片测试复现同类回读超时，同样零重复提交；③ 10 月 30 日在 v0.15.1 API 续传通路上（评估的 75b7abc 浏览器快照之外）的 YouTube 私享上传测试中，进程经历两次强制中断（首次在确认传输 8,388,608 字节后退出，第二进程保存视频 ID 后退出），第三进程完成缩略图并保持同一视频 ID，未产生重复视频。

Q6 [Reviewer #2 (Post-Publication Revision & Deletion — Algorithm 3)]
EN: How does Algorithm 3 prevent destructive compensation (such as delete-and-repost) during post-publication edits and deletions?
CN: 算法 3 是如何在发布后修订（vpub edit）与删除（vpub delete）中防止“先删帖再重发”这类破坏性操作的?

Standard SCI Rebuttal (English): In Saga terminology (Garcia-Molina & Salem, 1987), deleting and reposting a live social media item is not a clean compensating transaction because it destroys the post's URL, engagement history, and timestamp. Algorithm 3 (implemented in PRs #20, #21, #25, #29, and #30 at commit 75b7abc) checks expected_identity before any action and uses four exact status codes: EDIT_VERIFIED (live metadata already matches target; no save click issued), EDIT_MISMATCH (dry-run reports differences without saving), EDIT_UNAVAILABLE (on X, where the post menu exposes no native Edit action, the adapter stops without deleting, reposting, or replying), and EDIT_SUBMITTED_UNVERIFIED. For single-work deletion (vpub delete), the adapter requires a prior dry-run and explicit human approval of the target work, then refreshes the platform catalogue to confirm the work's absence.

核心防御逻辑：算法 3 严格对齐了真实代码中的四个状态码（EDIT_VERIFIED、EDIT_MISMATCH、EDIT_UNAVAILABLE、EDIT_SUBMITTED_UNVERIFIED）。特别是在 X 平台没有原生编辑按钮时，直接返回 EDIT_UNAVAILABLE 并终止，绝不删帖重发；删除作品则必须先经过 dry-run 与人工确认，并在删除后刷新列表确认该作品已不存在。

Q7 [Reviewer #3 (Operational Journal Evidence & Passive vs. Active Evidence)]
EN: In Table 3, why do you state that the 69-record operational journal shows 'absence of writes' rather than claiming it directly proves '68 suppressed writes'?
CN: 在表 3 中，为什么你们严格区分“日志记录中未发生未授权写入”与“主动拦截了 68 次越权尝试”?

Standard SCI Rebuttal (English): Because scientific honesty requires distinguishing passive outcome logs from active refusal logs (Sections 6.1 and 7.4). In data/operations-events.json (N=69 records; 46 timestamped across 25.3 hours from 26 Oct 01:27 to 27 Oct 02:45 UTC+8), the journal records what each checkpoint did, not what it refused: 20

records carry explicit zero-write counters (13 reads, 0 writes), 39 record platform_calls = 0, 1 records a read-only platform call without a write field (#22), 1 records the authorized platform write (#52: 4 reads, 1 write, replay = 0, new_risk = false), and 8 early records carry no call/write counter fields. While this outcome log is fully consistent with the gate, direct proof that planRun blocks unauthorized callers, stop flags, and unscheduled jobs comes from the 10 unit tests in tests/workflows.test.mjs (10/10 passed) and the scheduled heartbeat evidence.

核心防御逻辑：这是主动化解审稿人质疑的绝佳设计！运维日志记录的是“每次检查的结果（仅 #52 写了 1 次，其余 0 写入）”，而不是“拦截了多少次恶意攻击”。我们在表 3 按 JSON 真实字段分成 20 / 39 / 1 / 1 / 8 五类（总计 17 次平台读取、1 次写入），同时指出门控的主动拦截逻辑由 workflows.test.mjs（10/10 通过）直接证明，并在未来工作中提出增加“拒绝尝试日志”。这种严谨度让审稿人挑不出任何毛病！

Q8 [Reviewer #3 (Software Scale, Bundled Code Disclosure & Snapshot Pointers)]
EN: How are the code and test counts in Table 4 computed, and how do you handle bundled third-party code and repository evolution?
CN: 表 4 中的代码与测试规模是如何计算的？你们是如何处理内置第三方代码与仓库版本演进的?

Standard SCI Rebuttal (English): All counts in Table 4 are computed at the evaluated head commit 75b7abc (3 November 2026) using physical line counts (via paper/reproduce_paper_tables.mjs) and compared transparently against the v0.16.0 release tag (commit a46bd19: 107 source files, 84 test files, 2,009 test functions). At 75b7abc, publishops-crosscast contains 123 Python source files (26,972 physical lines), of which 42 files (8,967 lines) are an unmodified bundled copy of MIT-licensed bili-cli v1.2.1 (disclosed in NOTICE.md) and 81 files (18,005 lines) constitute CrossCast's own package. The test suite comprises 126 files (122 test_* modules + 4 helper files, 37,029 physical lines, 2,737 def test_* functions), yielding a test-to-source line ratio of 1.37 overall and 2.06 against CrossCast's own package.

核心防御逻辑：表 4 不仅用物理行数精确复算了 commit 75b7abc（2026年11月3日快照）的全部代码，还主动披露其中 42 个文件（8,967 行）是原样内置的 MIT 开源组件 bili-cli v1.2.1，自研 CrossCast 包为 81 个文件（18,005 行）；对应 126 个测试文件（37,029 行，2,737 个测试函数），自研测试源码比高达 2.06，且同时给出了 v0.16.0（a46bd19）的对比数字，做到了 100% 透明。

Q9 [Reviewer #3 (Instagram Timing Interpretation & N=1 Scope)]
EN: In Table 7 and Figure 5, what does the 380.551 s (92.32%) Instagram step actually measure, and can you generalize that exact percentage to all platforms?
CN: 在表 7 和图 5 中，Instagram 的 380.551 秒（92.32%）具体测的是什么？这个百分比能直接推广到所有平台吗?

Standard SCI Rebuttal (English): As stated in Section 6.3 and Table 7 (N = 1 timed Instagram release: 423.236 s end-to-end, 412.217 s inside vpub publish), the 380.551 s post-Share phase spans from the single Share click until the platform returns confirmation. From outside the platform's servers, client telemetry cannot separate network media transfer completion, server-side transcoding/processing, and final confirmation within that 380.551 s window, and the phase breakdown selects major phases rather than an exhaustive additive split (leaving a 15.384 s derived residual inside vpub publish). This study does not claim the 92.32% figure transfers numerically to other platforms; rather, it empirically illustrates the structural hazard that motivates Layer 3: a long post-press in-doubt window where an unguarded timeout retry would press Submit a second time.

核心防御逻辑：我们在摘要、Highlights 与第 6.3 节明确说明：380.551 秒（92.32%）来自单次完整计时的 Instagram 发布（N=1），且从客户端视角无法把该窗口内的上传传输、服务端转码与确认拆开（命令内部其余

未拆分步骤派生余项为 15.384 秒）。我们不宣称“所有平台都是 92.32%”，而是用这一实测证据直观揭示“点击提交后存在漫长的存疑窗口”，从而论证在点击前先行落盘 UNKNOWN 状态的架构必然性。

Q10 [Reviewer #3 (Cohort Complete Separation & Exposure-Age Confounding)]
EN: In Section 6.5 and Table 10, why do you report descriptive quantiles for the 55 Bilibili records without running a significance test?
CN: 在第 6.5 节和表 10 中，为什么你们对 55 条 Bilibili 记录只报告分位数等描述统计，而不做显著性假设检验?

Standard SCI Rebuttal (English): In data/creator-video-snapshot.json (N = 55 records: n = 45 in human_led_history published Oct 2019–Oct 2026, and n = 10 in current_ai_series comprising 5 published videos aged 1.6 hours to 3.1 days and 5 scheduled releases), every single historical video has more views than every single newly published AI-series video: min(human_led_history) = 274 views (mean 40,884.31, SD 64,604.81, median 15,407.0, Q1–Q3 1,722–67,502, max 333,725) versus max(current_ai_series) = 231 views (mean 133.20, SD 99.53, median 183.0, Q1–Q3 42–199, min 11). Because the two distributions exhibit complete separation (274 > 231) across a multi-year versus multi-hour exposure gap, a rank test on views mechanically yields U = 0 and would merely restate exposure age rather than content quality. Furthermore, weekly play growth (76,162 vs. 21,398) was led by older travel vlogs and 88.09% TV-client traffic, reinforcing why raw channel totals must not be attributed to the new AI workflow.

核心防御逻辑：在 55 条快照中，45 条人类历史视频最低播放量是 274（Q1–Q3 为 1,722–67,502），而 5 条刚发布 1.6 小时–3.1 天的 AI 视频最高播放量是 231（Q1–Q3 为 42–199；274 > 231，两组样本完全不重叠，即 Complete Separation，若硬做秩检验 U 恒等于 0）。而且周播放增长（76,162 vs 21,398）主要来自电视端（88.09%）推荐的老旅行视频。因此不做误导性的显著性检验、如实呈现曝光时长混杂，恰恰是第 4 层“认知治理（Epistemic Governance）”的核心示范！

Q11 [Reviewer #3 (Analytical Comparison vs. Executed Ablation — Table 11)]
EN: How are the baseline alternatives in Table 11 evaluated if you did not intentionally run unguarded agents on live public accounts?
CN: 既然你们没有在真实公开账号上故意运行不设防的基线智能体，表 11 中的基线对比是如何评估的?

Standard SCI Rebuttal (English): Table 11 is explicitly framed as an analytical comparison rather than an executed ablation trial. For the unguarded baselines, the table states what would occur under each removed layer based on architectural mechanism ('reasoned, not executed'), while for our four-layer design every cell explicitly tags its actual evidence source: field traces (e.g., the two Douyin read-back timeouts and the Instagram login escalation), automated test suites (e.g., workflows.test.mjs and the 2,737 CrossCast test functions), review logs (audio/loudness defect detection in Table 8), or code inspection.

核心防御逻辑：表 11 明确界定为“分析性对比（Analytical Comparison）”，将未执行的基线标注为基于机制推导的后果（would occur），而本文方案的每一行都严格标注了实证来源（现场记录 Field、自动化测试 Test、审校记录 Review 或代码实现 Code），不把机制推导伪装成跑过的消融实验。

Q12 [Reviewer #2 (Advisory Scheduling Gate vs. Hard Platform Enforcement)]
EN: If planRun in Layer 1 is a pure JavaScript function, couldn't a rogue script or operator bypass it and call the release CLI directly?
CN: 既然第 1 层的 planRun 是一个纯函数，如果某个脚本绕过它直接调用发布 CLI 怎么办?

Standard SCI Rebuttal (English): Section 4.2 explicitly notes this defense-in-depth boundary: planRun is an advisory scheduling gate for the autonomous runner, whereas hard enforcement at the platform boundary resides in Layers 2 and 3 inside publishops-crosscast itself. Even if a caller bypasses planRun, vpub publish refuses to execute a live write without a matching --confirm CURRENT_PLAN_HASH digest (Layer 2) and an unlatched receipt lock (Layer 3).

核心防御逻辑：第 4.2 节坦诚说明了这一纵深防御边界：Layer 1（planRun）负责约束后台定时调度器，而真正卡死外部平台写入边界的是内置在 vpub publish 命令内部的 Layer 2（--confirm 计划哈希锁）与 Layer 3（进程锁与回执状态机）。即使绕过 Layer 1，没有匹配的计划哈希与未锁定回执也无法触发发布。

Q13 [Reviewer #2 (Attestation Boundary of --person-approved)]
EN: Does the --confirm digest cryptographically prove that a human being, rather than an agent, typed the command?
CN: --confirm 摘要能否在密码学意义上证明敲下命令的一定是人类而不是智能体?

Standard SCI Rebuttal (English): No, and Section 4.3 states this boundary explicitly: the --confirm CURRENT_PLAN_HASH digest cryptographically binds the approval to the exact metadata and media SHA-256 content reviewed during dry-run (preventing TOCTOU content drift), whereas the --person-approved flag is the caller's attestation that human review took place, as documented in the CLI's own help text.

核心防御逻辑：第 4.3 节如实界定了密码学边界：--confirm 哈希在数学上保证的是“实发内容与预演审阅内容字节级一致（防 TOCTOU 篡改）”，而“是否由真人批准（--person-approved）”则是调用方的声明与操作规程边界。

Q14 [Reviewer #1 (Single-Case Generalizability & External Validity)]
EN: What generalizes from a single-organization, nine-day study to other software engineering and project management teams?
CN: 从单一机构、9 天现场部署的研究中，有哪些结论可以推广到其他软件工程与项目管理团队?

Standard SCI Rebuttal (English): Following Flyvbjerg (2006), Runeson & Höst (2009), Wieringa (2014), and Wieringa & Daneva (2015), generalization proceeds by architectural mechanism ('analytical generalization', Section 7.4). While platform-specific latency numbers (such as the 380.551 s Instagram wait) are not claimed to transfer numerically, the underlying structural dilemma transfers to any project where AI agents act on external APIs or browser endpoints that lack idempotency keys and transactional rollback: write-ahead UNKNOWN receipt persistence before the final press, content-addressed approval tokens, and identity-bound in-place revision provide a reusable reference architecture.

核心防御逻辑：依据 Flyvbjerg (2006) 与 Wieringa & Daneva (2015) 的“按机制分析性推广（Analytical Generalization）”，具体平台的秒数无需照搬，但只要外部接口缺乏幂等键与回滚机制，“点击前预写 UNKNOWN 回执 + 内容寻址计划哈希 + 原位身份核验修订”这一参考架构就具有广泛适用性。

Q15 [Editor-in-Chief (Future Research & Measurable Gate Suppression)]
EN: What is the most important next step to strengthen this governance framework in future empirical deployments?
CN: 在未来的实证部署中，进一步强化该治理框架最关键的下一步工作是什么?

Standard SCI Rebuttal (English): As outlined in Section 8, the top priority is to log gate decisions and refused attempts (not only executed outcomes) so that suppression rates become directly measurable, alongside timing bounded concurrent multi-platform runs against sequential execution, running the planned blinded identical-task multi-model

comparison with per-call cost logging, and replicating the architecture across independent teams.

核心防御逻辑：第 8 节明确提出了 4 项紧扣研究局限的未来工作：① 不仅记录执行结果，还要持久化记录每一次门控决策与被拒绝的尝试，使“拦截率”可直接测量；② 实测受控并发发布与串行发布的性能对比；③ 执行带成本记录的多模型同题盲测；④ 在外部独立团队中开展跨机构复现。